

DESIGN PATTERN– Introduction

Mamadou Bousso
Université de Thiés

DESIGN PATTERN-

Introduction

Plan

Introduction

- • Objectifs du cours
- • Contenu du cours
- • Introduction à l'architecture logicielle
- Historique

Définitions

Pourquoi l'architecture logicielle

Styles, patrons et idiomes

Approche traditionnelle

INTRODUCTION



Objectifs

Comprendre et mettre en pratique des patrons de conception

- Acquérir et maîtriser une approche méthodologique pour analyser et concevoir un logiciel selon des principes architecturaux;
- Concevoir un logiciel orienté objet réutilisable;
- Définir et présenter des modèles de conception

INTRODUCTION A L'ARCHITECTURE LOGICIELLE

Définitions

- Génie logiciel (Software engineering)
 - L'application systématique et disciplinée d'approches quantifiables de développement et de maintenance de logiciels, et l'étude de ces approches. (IEEE 610.12)
- Architecture logicielle (Software architecture)
 - L'architecture logicielle d'un programme ou d'un système est la structure ou les structures du système, comprenant les composantes logicielles, les propriétés exposées de ces composantes et les interactions entre celles-ci. (Bass, Clements et Kazman 2003)

Définitions

L'architecture logicielle est une abstraction d'un logiciel

- Collection de composantes

Client, serveur, filtre, couche, contrôleur, agent

Topologie de composantes (relations)

- Agrégation, classification, généralisation

Interactions entre les composantes

- Connecteurs
 - Propriétés des composantes
- Mode d'interaction
 - Par ex. RPC, Partage de mémoire, synchronisation



Définitions



Composantes



Perspective orientée objet

Un module (package, librairie)

Un groupe de classes

Encapsulation des fonctions



Perspective procédurale

Un groupe de fonctions et de structures de données



Connecteurs : La signature des composantes

La définition des interfaces, des services offerts par une composante

Les fonctions et les propriétés des composantes

Pourquoi un architecte?

- Satisfaire les besoins du propriétaire
- Assurer la longévité de la maison (revente, charpente, catastrophe)
- Planifier pour permettre les extensions possibles
- Faciliter l'entretien et le confort
- Contenir les coûts
- Guider les travailleurs qui vont la construire

Pourquoi un architecte logiciel?

- Assurer la stabilité et la robustesse du système
- Permettre une maintenance relativement facile
- Permettre les évolutions futures
- Contenir les coûts
- Minimiser les complications
- Guider les programmeurs dans leurs efforts de développement

Une architecture logicielle

Des questions importantes auxquelles l'architecture doit répondre:

- **Quelle est la nature des composantes?**
 - Des processus, des programmes ou les deux?
 - Comment sont organisées les composantes?
 - Comment sont déployées les composantes?
- **Quelles sont les responsabilités des composantes?**
 - Que font les composantes?
 - Quelle est leur rôle dans le système?

Une architecture logicielle

Comment les composantes communiquent-elles?

- Messages, Appels, Synchronisation, Partage, Négociation, ... ?
- Quels sont les mécanismes de communication utilisés?
- Quelles informations sont transmises par ces mécanismes?

Comment sont organisés les composantes?

- Topologie

Une bonne architecture

- Indique le partitionnement d'un système en sous parties;
- Spécifie ce qui fait fonctionner les parties ensemble;
- Donne une vision globale (i.e. de haut niveau et non détaillée) de la conception du système;

Une bonne architecture

Communique une vision du logiciel qui est partagée et comprise par les différents intervenants du projet

Indique les moyens retenus pour atteindre les objectifs en terme de qualité

Communique aux développeurs ce qui doit être concrétisé

Architecture vs Conception

L'architecture est une forme de conception

- Les conceptions ne sont pas toutes architecturales
- Plusieurs détails de bas niveau ne sont pas spécifiés dans une architecture
 - Ex. Le choix d'une structure de données
 - Ex. Le choix d'un algorithme
- L'architecture ne définit pas une implémentation du logiciel
- Mise plutôt sur la structure et moins sur le domaine d'application
 - Les fonctionnalités sont orthogonales à la structure



Styles, Patrons ou modèles de conception et Idiomes

Termes fréquemment utilisés pour décrire

- Les structures utilisées pour la conception de logiciel
- Des recettes éprouvées et reconnues des praticiens
- Des exemples de bonnes pratiques
 - Par opposition aux mauvaises pratiques (antipatterns)
- Des trucs et astuces
- Des spécificités utiles de certaines technologies

Styles, Patrons et Idiomes

Styles (macro patterns)

- Structures de haut niveau pour décrire un logiciel d'un point de vue global
- Ex. Modèle par couche (layered), MVC, ...

Patrons (micro patterns)

- Solutions abstraites applicables à différents contextes mais qui offrent les mêmes avantages chaque fois
- Correspond aux Design Patterns

Idiomes

- Décrit de bonnes pratiques liées
 - à un formalisme de programmation
 - à un langage en particulier

Documentation - UML

- Notation préconisée en orienté objet
- Utilise plusieurs diagrammes pour décrire une composante ou un système
- Ces diagrammes permettent de répondre rapidement et de façon standardisée à plusieurs questions

Conclusion

L'architecture est une conception de haut niveau d'un logiciel

Beaucoup de recettes sont déjà disponibles

Ne pas réinventer la roue!

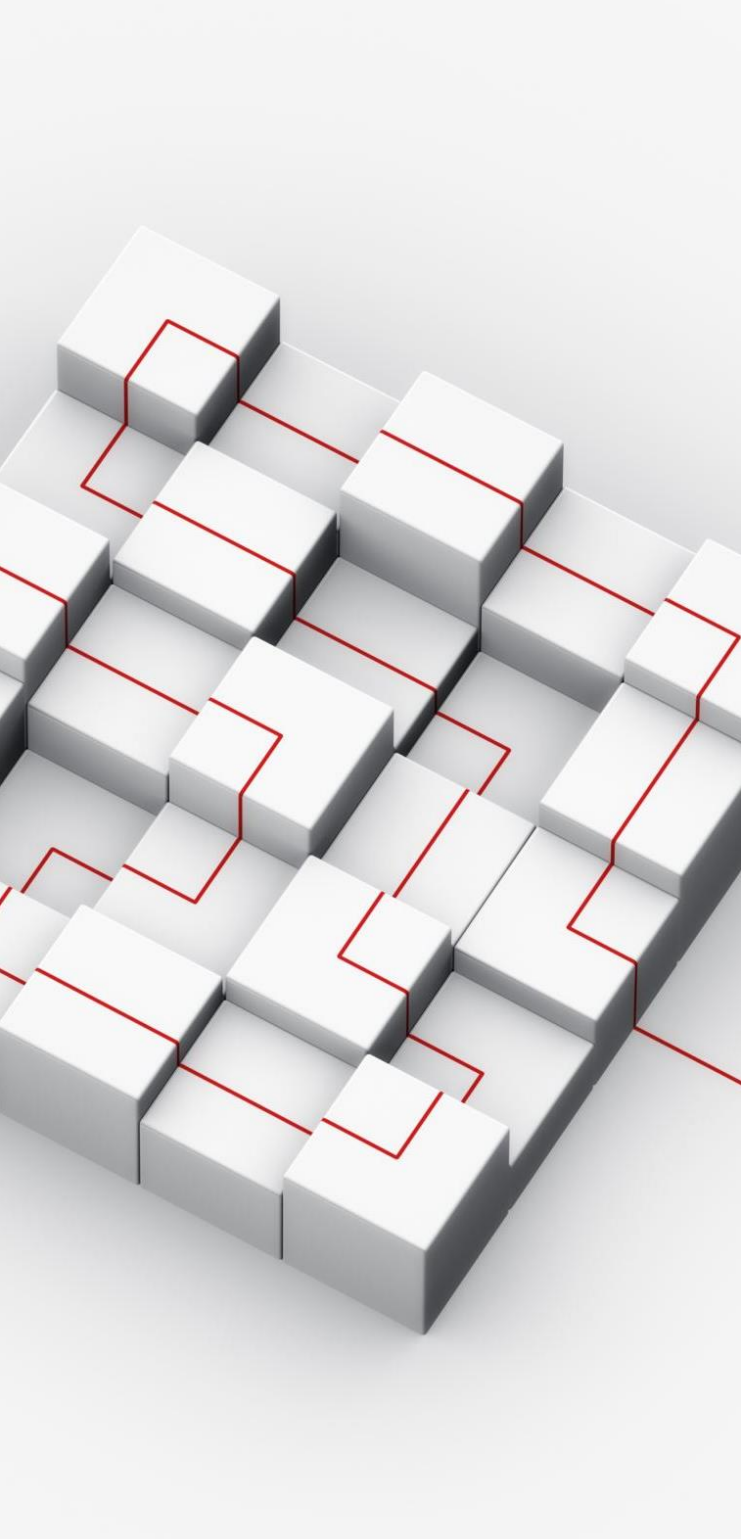
La conception architecturale est un élément important de l'approche orientée objet

L'architecture repose moins sur les fonctions que la qualité du logiciel

Lecture Architecture

https://fr.wikipedia.org/wiki/Architecture_logicielle





Introduction aux modèles de conception

Points importants: les modèles de conception

D'après Christopher Alexander:

« chaque modèle décrit un problème qui se manifeste constamment dans notre environnement, et donc décrit le cœur de la solution de ce problème, d'une façon telle que l'on peut réutiliser cette solution des millions de fois, sans jamais le faire deux fois de la même manière »

Qu'est-ce qu'un modèle de conception?

Il est composé de quatre éléments essentiels:

- Un **nom** qui permet de décrire en un ou deux mots un problème de conception, ses solutions et leurs conséquences;
- Un **problème** qui décrit les situations où le modèle s'applique;
- Une **solution** qui décrit les éléments qui constituent la conception, les relations entre eux, leurs parts dans la solution, leur coopération;
- Les **conséquences** qui sont les effets résultants de la mise en œuvre du modèle et les variantes de compromis que celle-ci entraîne.

Cas du génie logiciel

Les conséquences sont liées à:

- des compromis d'encombrement mémoire et de rapidité d'exécution;
- Des aspects du langage et du codage;
- La flexibilité du système;
- L'extensibilité du système;
- La portabilité du système.



Description d'un modèle de conception

Nom du modèle: il contient succinctement son principe;

Intention:

- Que fait effectivement le modèle?
- Quelle est sa raison d'être?
- Quel problème résout-il?

Alias: autres noms reconnus du modèle;

Motivation: scénario qui illustre un cas de conception et qui montre comment la structure de classe et d'objet du modèle contribuent à la solution de ce cas.

Description d'un modèle de conception

Indications d'utilisation:

- Quels sont les cas qui justifient l'utilisation du modèle?
- Quelles situations de conception peu satisfaisantes peuvent tirer avantage de l'utilisation du modèle?
- Comment reconnaître ces situations?

Structure: représentation graphique des classes du modèle basée sur UML;

Constituants: classes et objets intervenant dans le modèle avec leurs responsabilités;

Description d'un modèle de conception

Collaborations: comment les constituants collaborent-ils pour assumer leurs responsabilités?

Conséquences:

- comment le modèle de conception assument-ils ses objectifs;
- Quels compromis implique son utilisation et quel en est l'impact?
- Quelles parties de la structure d'un système permet-il de modifier?

Implémentation: de quels pièges astuces, ou techniques faut-il être averti lors de l'implémentation du modèle;

Description d'un modèle de conception

Exemples de code: extraits de programme qui illustrent qu'il convient de développer le modèle dans un langage donné?

Utilisations remarquables: exemples de modèles appartenant à des systèmes existants

Modèle apparentés:

- Quels modèles de conception sont en relation étroite avec celui traité?
- Quelles sont les différences importantes?
- Avec quel autre modèle peut-il être employé?

Modèle de base

Concepts importants qui reviennent dans les patrons plus complexes

Vocabulaire de base

Certains sont partie intégrante de plusieurs langages de programmation orientée objet

Modèle de base

Interface & Marker Interface

- Encapsuler la signature d'un service offert par plusieurs classes
- Marquer une classe d'un signe distinctif

Abstract Parent Class

- Permettre différentes définitions de services pour des classes similaires

Private Methods

- Limiter l'accès au comportement interne d'une classe

Modèle de base

Accessor Methods

- Contrôler l'accès aux propriétés des instances d'une classe

Immutable Objet

- S'assurer que l'état d'une instance ne peut être modifiée

Constant Data Manager

- Regrouper dans une classe centralisée les constantes d'une application

Interface

Problème

- Un même service peut être offert par plusieurs classes
- On ne veut pas lier le client à une classe concrète

Solution

- Définir une interface spécifiant le service offert (Abstraction)
- Les classes serveur implémentent cette abstraction
- Les clients réfèrent au type de l'interface et travaillent avec une instance d'une classe implémentant cette interface

Conséquences

- Améliore la maintenabilité
- Améliore l'adaptabilité
- Si sur-utilisation Perte de performance

Interface-Exemple

Algorithme de recherche dans une liste

- Plusieurs algorithmes possibles (Séquentielle, Binaire)
- L'interface IRecherche définit la signature du service
- Le service est implanté par les classes RechercheSequentielle et RechercheBinaire
- Le client travaille avec une référence de type IRecherche

Marker Interface

Dans certains cas, il est utile de pouvoir marquer une classe d'un signe distinctif indiquant certaines caractéristiques de la classe.

L'interface de marquage ne définit aucune méthode

Peut également s'exprimer par d'autres mécanismes selon le langage

- Ex. Annotations, Propriétés
 - Ex. en Java l'interface cloneable indique que la classe supporte totalement la méthode clone
- Véritable clonage

Abstract Parent Class

Problème

- Un service implanté par différentes classes diffère sur certaines parties

Solution

- Définir une classe abstraite qui sert de base
 - Parties communes Méthodes concrètes
 - Parties variables Méthodes abstraites
- Redéfinir les parties variables dans des sous-classes (spécialisation).

Abstract Parent Class

Problème

- Un service implanté par différentes classes diffère sur certaines parties

Solution

- Définir une classe abstraite qui sert de base
 - Parties communes Méthodes concrètes
 - Parties variables Méthodes abstraites
- Redéfinir les parties variables dans des sous-classes (spécialisation).

Classe abstraite

- Classe qui contient une ou plusieurs méthodes abstraites
- Peut contenir des méthodes concrètes (implantées) qui sont la partie invariable de la classe

Méthode abstraite

- Méthode déclarée mais qui n'est pas implantée
- Les méthodes abstraites représentent la partie variable de la classe abstraite

Abstract Parent Class

Plusieurs sous-classes peuvent être définies pour implanter des comportements différents pour les méthodes abstraites

Une classe qui hérite d'une classe abstraite doit implanter toutes les méthodes abstraites de la classe parent

- Sinon, la sous-classe devient également abstraite

Abstract Parent Class

Une classe abstraite ne peut directement être instanciée

Les méthodes non abstraites du parent sont héritées

- Partie invariable redondante entre les classes

Note

- En Java, l'héritage multiple n'est pas permis.
- Donc, l'utilisation des interfaces est privilégié.
- Cependant, apporte de la redondance dans l'implantation des méthodes communes.

Abstract Parent Class

- Conséquences
 - Améliore la maintenabilité
 - Améliore l'adaptabilité
 - Si sur-utilisation Perte de performance
 - Java Pas d'héritage multiple
 - Héritage multiple Attention aux problèmes

Abstract Parent Class - Exemple

Comptes bancaires

- Compte épargne
 - Intérêt = 5 % par mois
 - Frais de transaction = 0.50\$
- Compte chèque
 - Intérêt = 1 % par mois
 - Frais de transaction = 0.15\$ si solde < 1000, sinon =0 \$



Abstract Parent Class - Exemple



Partie commune

Obtenir le solde – **double** `getSolde()`

Déposer un montant – **void** `deposer(double montant)`

Retirer un montant – **void** `retirer(double montant)`



Partie variable

Calculer les intérêts sur le solde – **double**
`calculerInterets()`

Calculer les frais de transaction – **double**
`calculerFraisTransaction()`



Les comptes spécialisés doivent implanter les



méthodes de la partie variable.



Private Methods



Permet de définir des méthodes utilitaires qui ne sont pas visibles aux clients qui utilisent la classe

```
public class ClasseM {  
    public void servicePublicA() {  
        // Service exposé  
    }  
    private void servicePrive1() {  
        // Service privé  
    }  
}
```

Accessor Methods

L'état d'un objet est défini par la valeur de ses différentes variables d'instance

- Variables d'instance publiques
- Variables d'instance privées

Les Accessor Methods permettent de contrôler

l'accès aux variables d'instance

Les Accessor Methods permettent de contrôler

les changements d'état de l'objet

- État sources vers État cible

Accessor Methods

Problème

- On veut contrôler l'accès aux variables d'instance de l'objet
- On veut contrôler les changements d'état de l'objet
- On veut limiter les problèmes de maintenance

Accessor Methods

Solution

- Rendre toutes les variables d'instance privées
- Définir des méthodes pour l'accès aux valeurs des variables d'instance
- Définir des méthodes de modification de l'état et des valeurs des variables d'instance
- Utiliser les méthodes d'accès et de modification
- même à l'interne de la classe

Accessor Methods

Conséquences

- Améliore la maintenabilité
- Limite le nombre de modifications si un attribut est modifié
- Permet le changement de représentation de l'état interne de la classe
- Cache si l'attribut est directe ou dérivé

Accessor Methods

Nomenclature des méthodes

- Variable non-booléenne
 - Lecture
 - type getXXX()
 - Écriture
 - void setXXX(type valeur)
- Variable booléenne
 - Lecture
 - boolean isXXX()
 - Écriture
 - void setXXX(boolean valeur)
 - void isXXX(boolean valeur)

Principes de conception numero 1

Encapsuler ce qui varie



**" Detecter les parties qui varient dans votre application
puis séparer-les de ce qui est statique "**



**Objectif: minimiser les effets causés par toute
modification**

Exercice

- Analyser le code des fichiers order.java et gestioncommande.java et separer ce qui varie de ce qui statique.

On suppose que le calcul des taxes devient plus complexe:

- Les taxes americaines sont calculées par zone(Amerique, Europe, Asie), par pays et par produit. Proposez un modèle qui respecte le principe défini dans la diapo precedent
-

Principes de conception numero 2

Programmer avec les interfaces et non avec les implementations



**" Programmer avec les interfaces et non avec les implementations.
Dependez des abstractions et non des classes concrètes "**



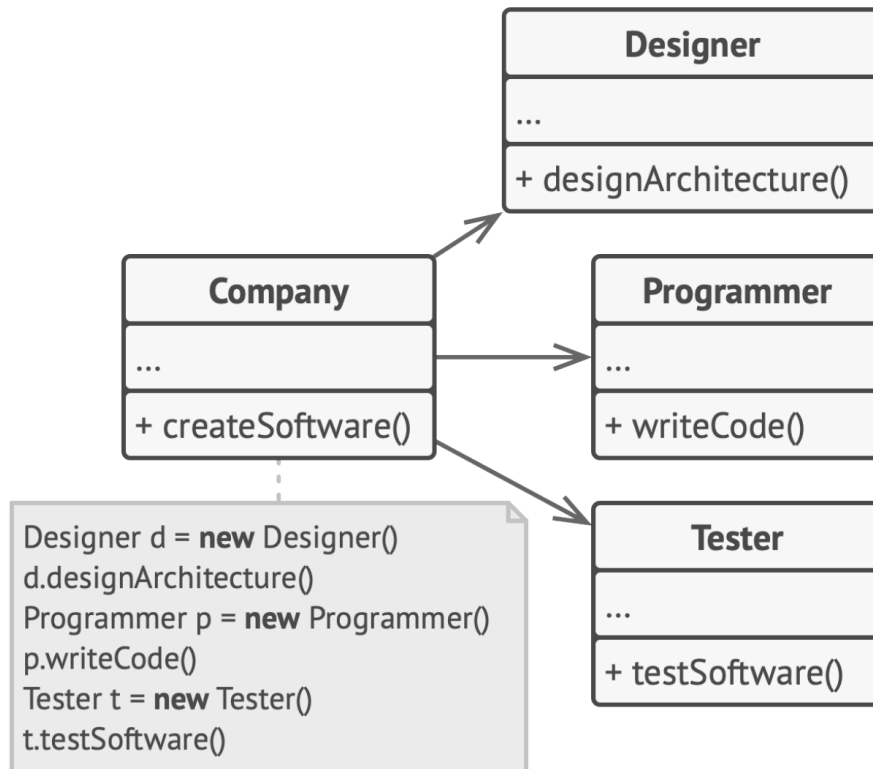
Objectif: rendre l'application flexible et evolutive



Methodologie

Lorsque vous voulez faire collaborer deux classes:

1. Déterminez exactement pourquoi un objet a besoin de l'autre : quelles méthodes exécute-t-il ?
2. Décrivez ces méthodes dans une nouvelle interface ou classe abstraite.
3. Faites implémenter cette interface à la classe qui est une dépendance.
4. Maintenant, rendez la seconde classe dépendante de cette interface, plutôt que de la rendre dépendante de la classe concrète.



Exercice: transformer ce design en appliquant le principe 2

Principes de conception numero 3



Favoriser la composition sur l'heritage



" Permet d'éviter tous les inconvénients de l'héritage "



Objectif: rendre l'application flexible et evolutive



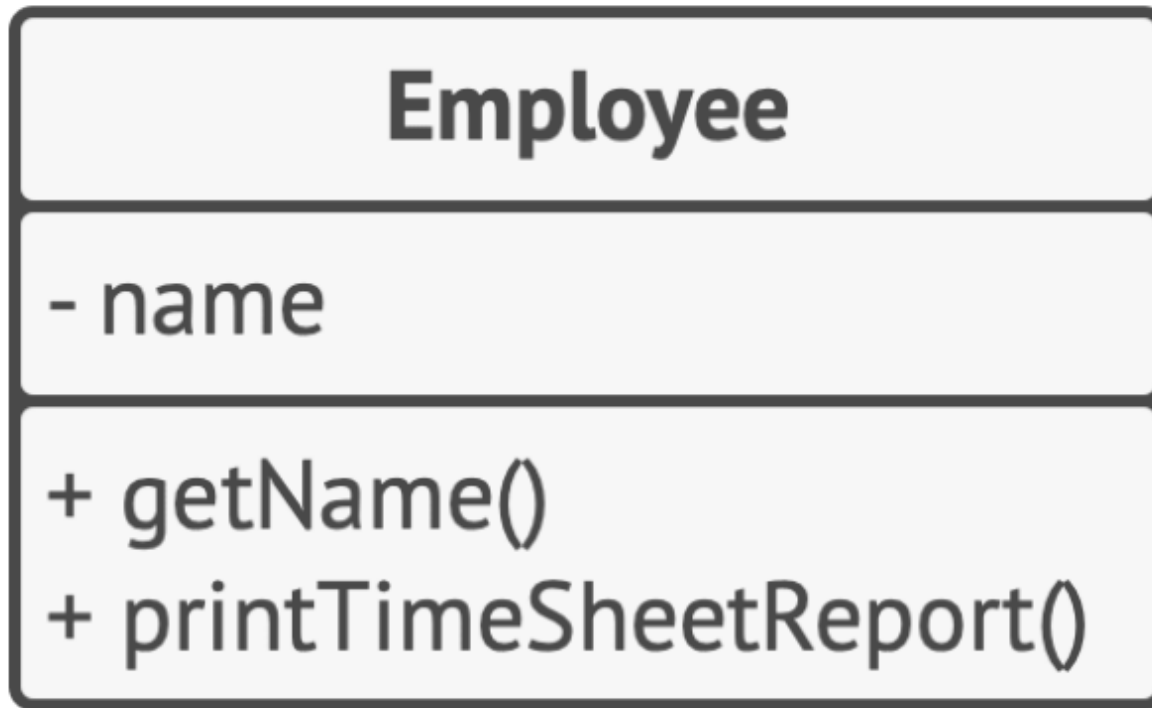
Les principes SOLID

- D'après Robert Martin: Agile Software Development, Principles, Patterns, and Practices
: <https://refactoring.guru/fr/principles-book>

Principe 1

UNE CLASSE NE DEVRAIT ÊTRE MODIFIÉE QUE
POUR UNE SEULE RAISON.

OBJECTIF: DIMINUER LA COMPLEXITE



Exemple

QUEL EST LE PROBLÈME AVEC CETTE CLASSE ? MODIFIER LA POUR LA RENDRE CONFORME AU PRINCIPE 1.



Open/close principe

"Une classe doit être ouverte à l'extension et fermée à la modification"

- Une classe est ouverte si vous pouvez l'étendre, créer une sous-classe
- Une classe est fermée (on peut également dire complète) si elle est prête à 100% à être utilisée par d'autres classes (son interface est clairement définie et ne sera plus modifiée dans le futur).

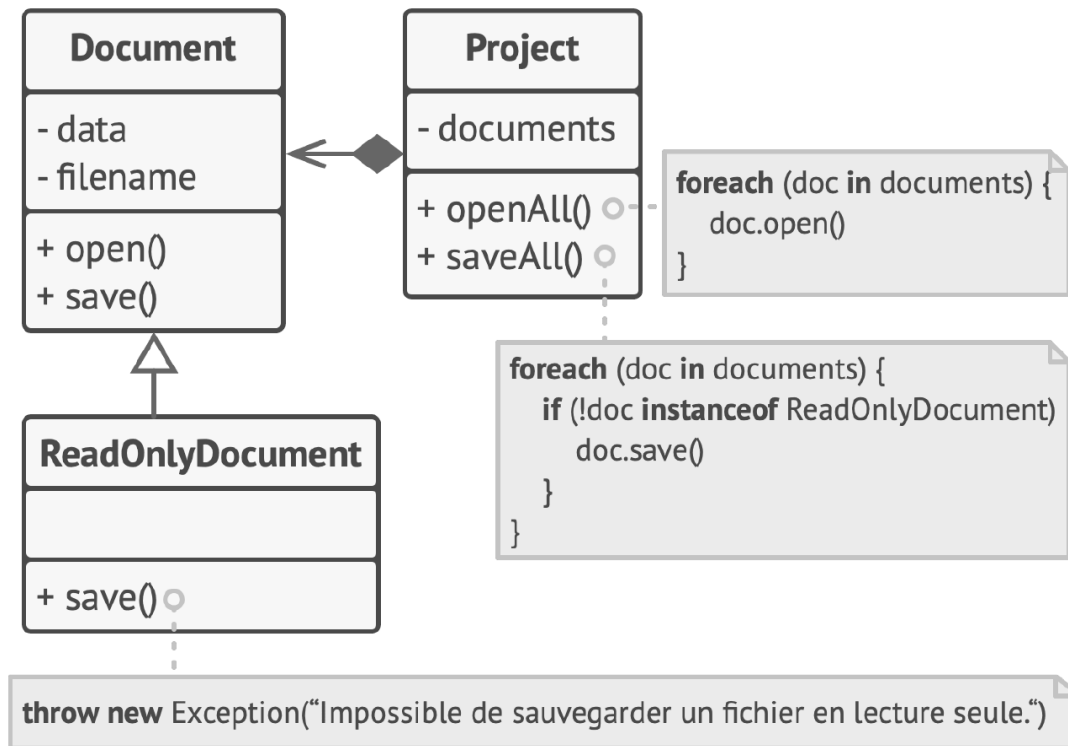
Objectif: éviter de créer des bugs quand vous ajoutez de nouvelles fonctionnalités

Principe de Liskov

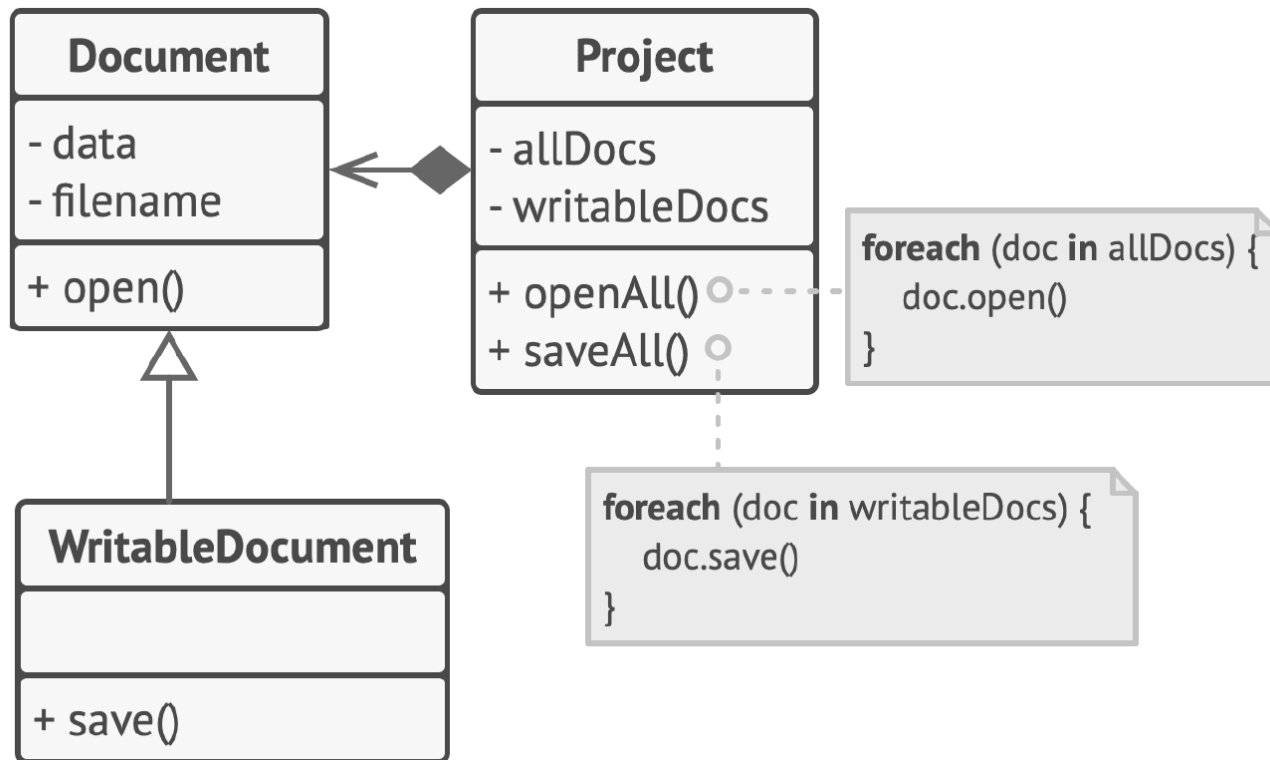
"Lorsque vous étendez une classe, rappelez-vous que vous devez être en mesure de passer des objets de la sous-classe à la place des objets de la classe mère sans faire planter le code."

Il s'appuie sur plusieurs règles:

- Les types des paramètres de la méthode d'une sous-classe doivent correspondre ou être plus abstraits que les types des paramètres dans la méthode de la classe mère.
- Le type de retour dans une méthode d'une sous-classe doit correspondre ou être un sous-type du type de retour de la méthode de la classe mère
- Une méthode dans une sous-classe ne devrait pas lever les types d'exceptions que la méthode de base n'est pas censée lever.
- Une sous-classe ne devrait pas affaiblir des postconditions
- Les invariants d'une classe mère doivent être préservés
- Une sous-classe ne doit jamais modifier les valeurs des attributs privés d'une classe mère.



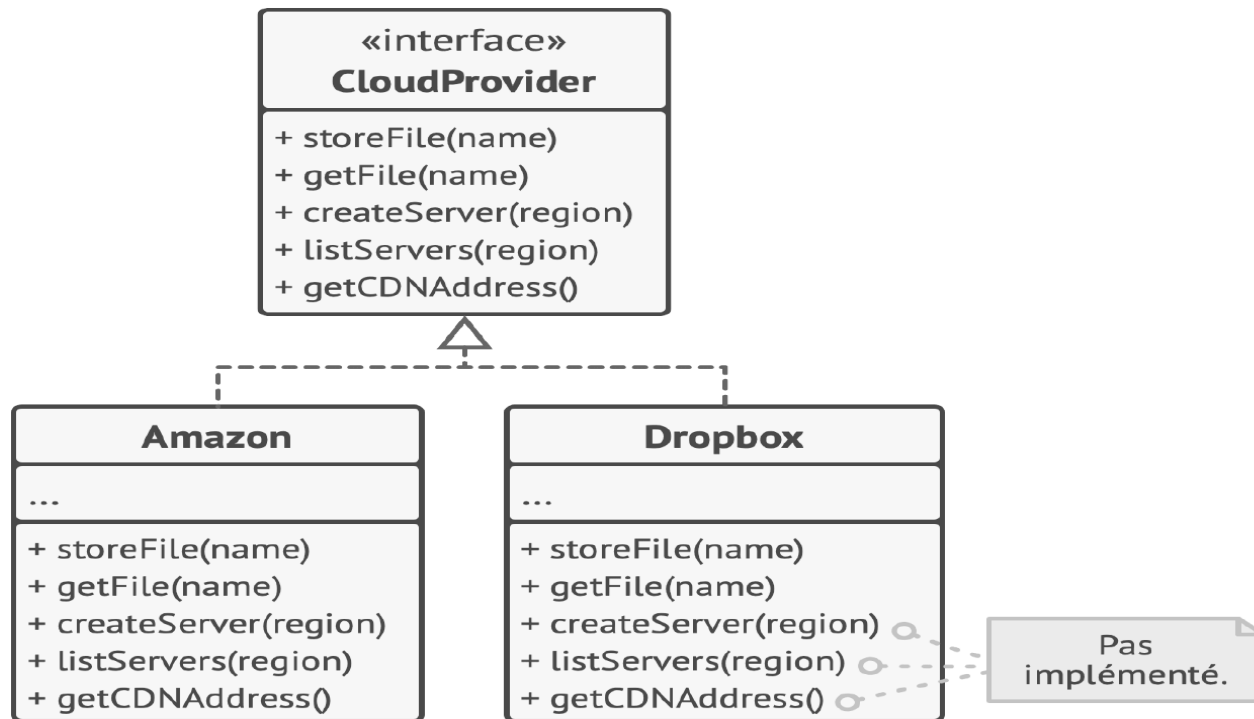
Exemple

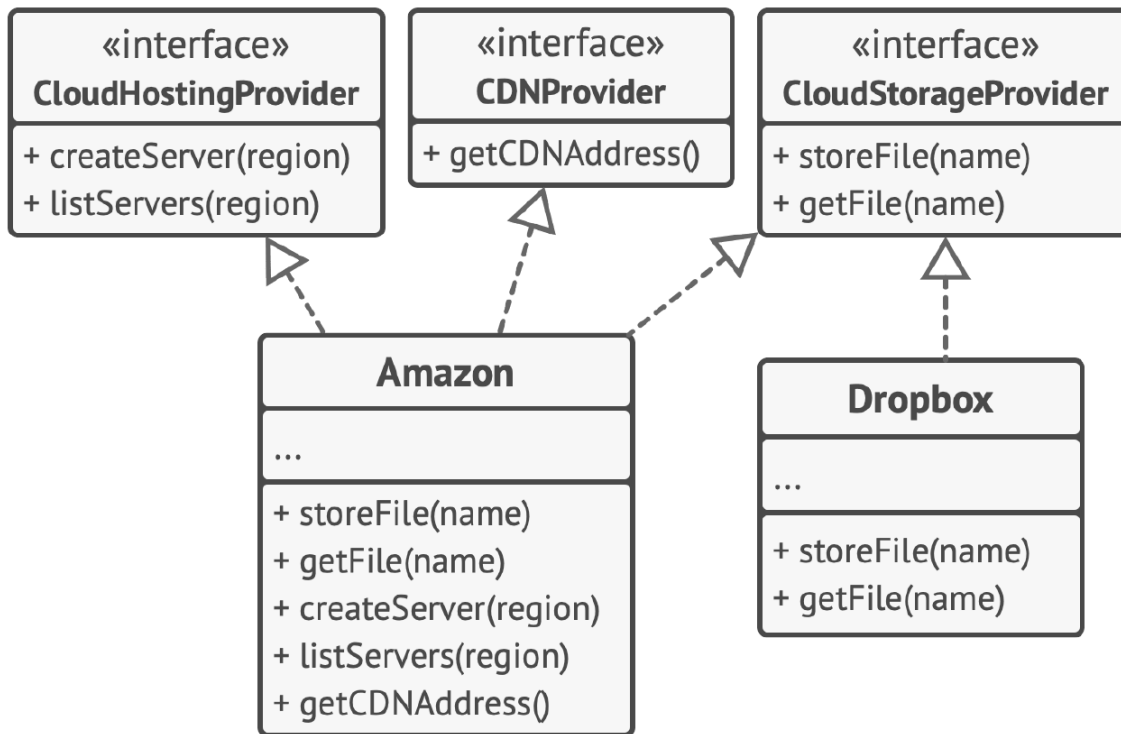


Correction

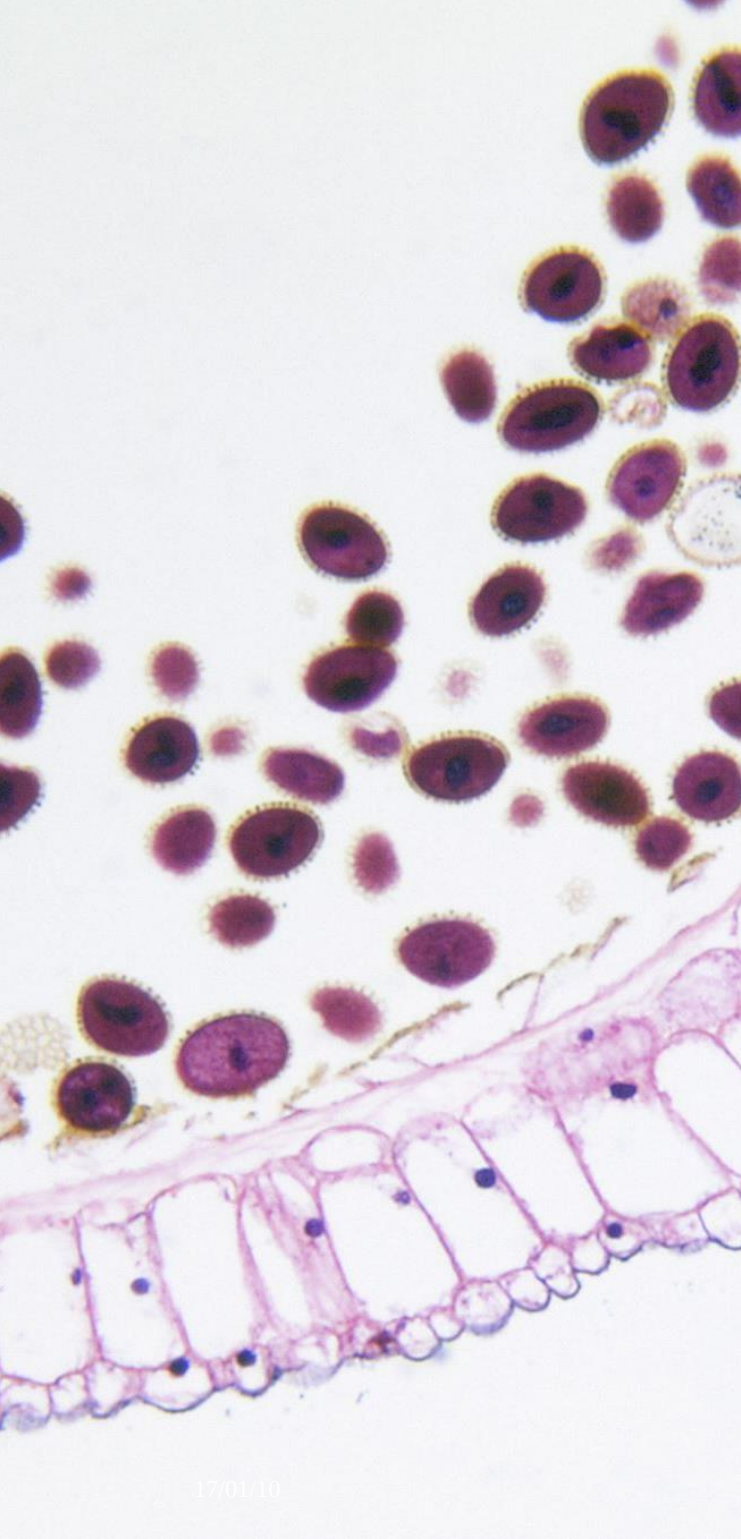
Principe de ségrégation des interfaces

"Les clients ne devraient pas être forcés à dépendre de méthodes qu'ils n'utilisent pas."





Principe de ségrégation des interfaces



Principe Inversion de dependance

"Les classes de haut niveau ne devraient pas dépendre des classes de bas niveau. Elles devraient dépendre toutes les deux d'abstractions. Une abstraction ne doit pas dépendre des détails. Les détails doivent dépendre de l'abstraction."

Les classes de bas niveau implémentent des traitements basiques comme utiliser le disque, transférer des données sur un réseau, se connecter à une base de données, etc.

Les classes de haut niveau contiennent la logique métier qui indique aux classes de bas niveau ce qu'elles doivent faire.

FIN
